MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 2

Software Design

Author: Ciordas Roxana-Andreea

Group: 30433

Faculty of Automation and Computer Science

March, 2026

Contents

1 Problem definition **2**

2 Analysis phase - Use cases & requirements **2**
2.1 Functional requirements 2
2.2 Use cases 3

3 Design phase **4**
3.1 Architecture overview 4
3.2 Entity-relationship diagram 5
3.3 Character Management Module 5
3.4 Authentication and Session Module 6
3.5 Design patterns 6
3.6 Class diagrams 7
3.7 Sequence diagrams 7

4 Implementation phase - Application **8**
4.1 Tools and technologies 8
4.2 Application structure 8
4.3 User manual 9

5 Testing and validation **10**

1 Problem definition

The primary objective of this project is to design and develop a desktop application that serves as a Dungeons and Dragons Character Creator System. The application provides a graphical user interface for users to build, edit, and store their characters. It features an authentication system, dividing users into Admins (who manage user accounts), Players (who have full Create, Read, Update, and Delete capabilities for their own characters), and Visitors (who possess read-only dashboard access). A core feature of the system is the point-buy stat allocator, which automatically calculates character attributes based on their chosen species and user-assigned bonus points. Under the hood, the application communicates with an SQLite database for data persistence and is structured around the Model-View-Controller (MVC) architectural pattern and emphasizing SOLID and GRASP principles.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

- **Authentication & Registration:** Users can register a new account using a valid email and password and log in to their account.
- **Role-Based Access Control:** The system supports three distinct user roles with specific permissions:
  - *Admins* can view the list of all registered users and delete user accounts, but cannot interact with characters.
  - *Players* can fully manage their own characters.
  - *Visitors* have read-only access to view the dashboard, create new characters that cannot be saved in the database.
- **Character Management:** The system allows Players to create, read, update, and delete characters associated with their account.
- **Stat Allocation (Point-Buy System):** The system applies automatically base statistical modifiers based on the user's chosen character Species. Furthermore, it must allow the user to distribute exactly 4 bonus points across six attributes (Strength, Dexterity, Constitution, Intelligence, Wisdom, Charisma) without dropping below the species' base stats.
- **Search and Sort:** Users are able to search for specific characters by name using a search bar and sort the displayed list either alphabetically by Name. Admins are able to search users by email.
- **View Details:** Users can select a character, after which they can view and edt them in the character editor.
- **Data Exporting:** Users can download a filtered and sorted list of their characters. The system supports multiple export formats: CSV, JSON, and XML.

- **Event Notifications:** The system implements an event-driven mechanism that listens for state changes. Whenever a character is created, updated, or deleted, the system automatically sends an email notification to the user.

Non-Functional Requirements

- **Security:** The application validates email formats using Regular Expressions and enforces a minimum password length of four characters during user registration. Furthermore, to protect against SQL injection vulnerabilities, all database queries and transactions are securely executed using Java's PreparedStatement objects.
- **Maintainability & Architecture:** The codebase strictly follows the Model-View-Controller (MVC) architectural pattern. It enforces the Single Responsibility and Information Expert principles by completely decoupling the graphical controllers from the database layer. All business rules, validations, and data routing are handled by dedicated Service classes (UserService, CharacterService), ensuring the codebase is modular and easy to extend.
- **Usability:** The JavaFX graphical user interface delivers intuitive visual feedback. As users allocate character attributes using the point-buy system, the remaining points counter updates dynamically without requiring manual calculation. Additionally, the system utilizes JavaFX dialog boxes to display error messages.
- **Data Persistence:** The system maintains data persistence through a SQLite database.

2.2 Use cases

The application support 3 types of "actors":

- **Admin:**That can perform actions concerning other users.
- **Player:** That can fully manage their characters, including creating, editing, viewing, and deleting characters, as well as allocating ability points.
- **Visitor** That have a read-only access permissions to the dashboard, allowing them to explore the application's interface without the ability to create, modify, or delete any data.

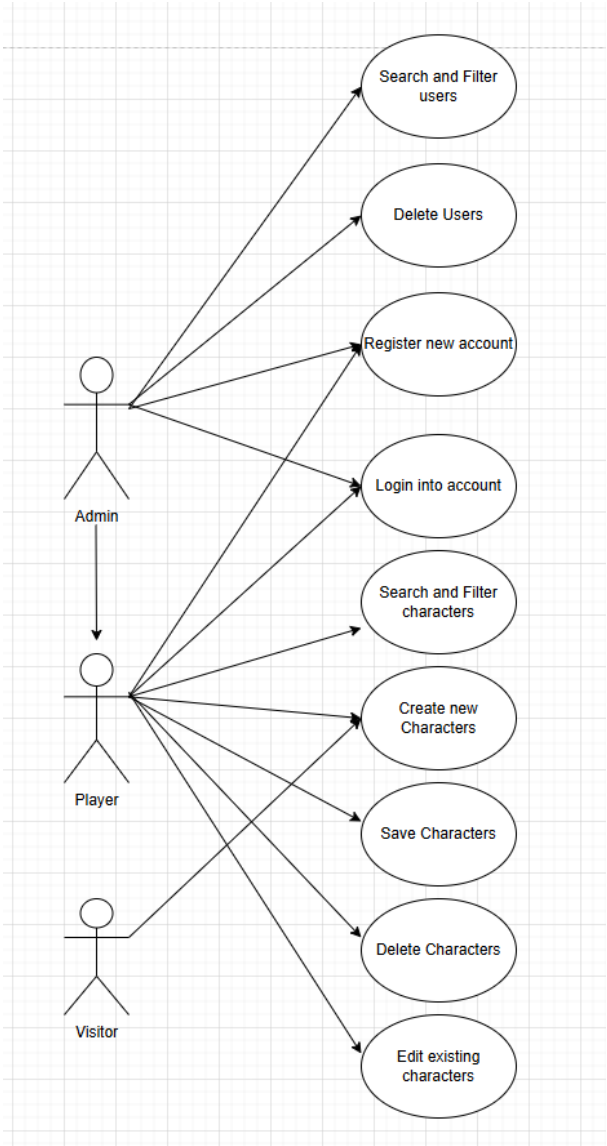

Figure 1: Use Case Diagram

3 Design phase

3.1 Architecture overview

The D&D Character Management System is built as a standalone desktop application utilizing a Layered Model-View-Controller (MVC). This architectural style which has been further refined into distinct, event-driven modules.

- **Model Package:** It contains the data structures, database repositories, and business logic services. It includes entity classes, alongside service classes that enforce domain rules and handle SQLite data persistence.
- **View:** This component defines the graphical user interface and visual layout of the application. It consists of JavaFX FXML files without containing any underlying logic.

- **Controller:** This package includes Java classes that ensure the communication between the Views and the Model. They capture user inputs from the GUI, pass requests to the Service layer for processing, and update the screen with the resulting data.
- **Resource Management Module:** Handles the core business rules and database persistence for characters (e.g., `CharacterService`).
- **Notification Service Module:** Operates completely independently, listening for system events and triggering user alerts.
- **Export Module:** Encapsulates the logic for formatting and downloading user data.

3.2 Entity-relationship diagram

The appliction utilizes 2 main database tables:

- **Users:** Contains all the registered users and their information.
- **Characters:** Contains all the characters created by the users.

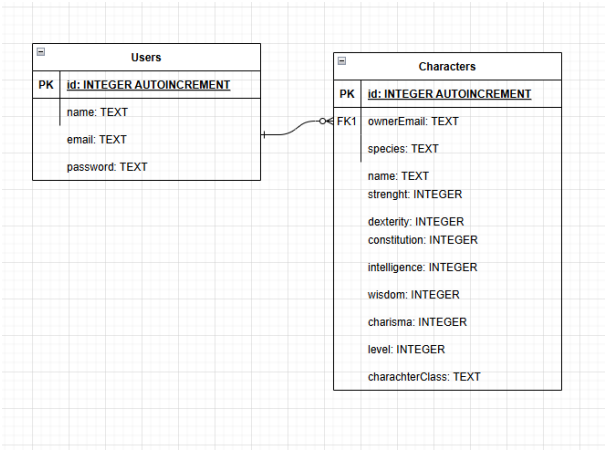

Figure 2: Entity-relationship diagram

3.3 Character Management Module

The Character Management Module is the core functional component of the application, responsible for the entire lifecycle the characters.

The primary responsibilities of this component include:

- **Characters Statistics Logic:** It ensures that character statistics follow the "Point-Buy" rules, preventing users from spending more than the 4 bonus points allowed or dropping below species-based minimums.
- **Data Manipulation:** It manages the CRUD operations by coordinating requests between the UI controllers and the database repository.
- **Data Filtering:** It utilizes Java Streams to perform real-time searching and categorical sorting of character lists, ensuring the UI stays responsive as the database grows.

3.4 Authentication and Session Module

The Authentication and Session Module is a critical architectural component consisting of the UserService, UserRepository and the UserSession.

- **Identity Management:** It handles the registration of new accounts and the verification of existing credentials.
- **Role-Based Access Control:** This module defines the permissions for different actors (Admin, Player, Visitor).
- **Session State Persistence:** It provides a globally accessible point of reference for the currently logged-in user.

3.5 Design patterns

This subsection outlines the specific design patterns implemented to ensure the application remains modular, scalable, and easy to maintain.

- **Observer Pattern:**
  - **Problem solved:** Facilitates event-driven communication between the Resource Management Module and the Notification Service Module without tightly coupling them. The CharacterService publishes events when character data changes. The NotificationService listens for these events to trigger email notifications.
  - **Classes involved:** EventPublisher, CharacterEventListener, CharacterEvent, and NotificationService.
- **Strategy Pattern:**
  - **Problem solved:** Encapsulates the logic for formatting and downloading user data. It allows the application to switch between different data export formats at runtime without modifying the core UI or controller logic.
  - **Classes involved:** ExportStrategy interface, CsvExporter, JsonExporter, XmlExporter.
- **Repository Pattern:**
  - **Problem solved:** Decouples the business rules inside the Resource Management Module from the SQLite persistence layer. It provides a centralized interface for all database operations, ensuring the CharacterService does not contain direct SQL queries.
  - **Classes involved:** UserRepository and SqlCharacterRepository.
- **Singleton Pattern:**
  - **Problem solved:** Ensures a single global point of access. It is used for the UserSession to prevent passing user objects manually between controllers, and for the DatabaseManager to guarantee only one secure SQLite database connection is opened and utilized by all modules during the application's lifecycle.

3.6 Class diagrams

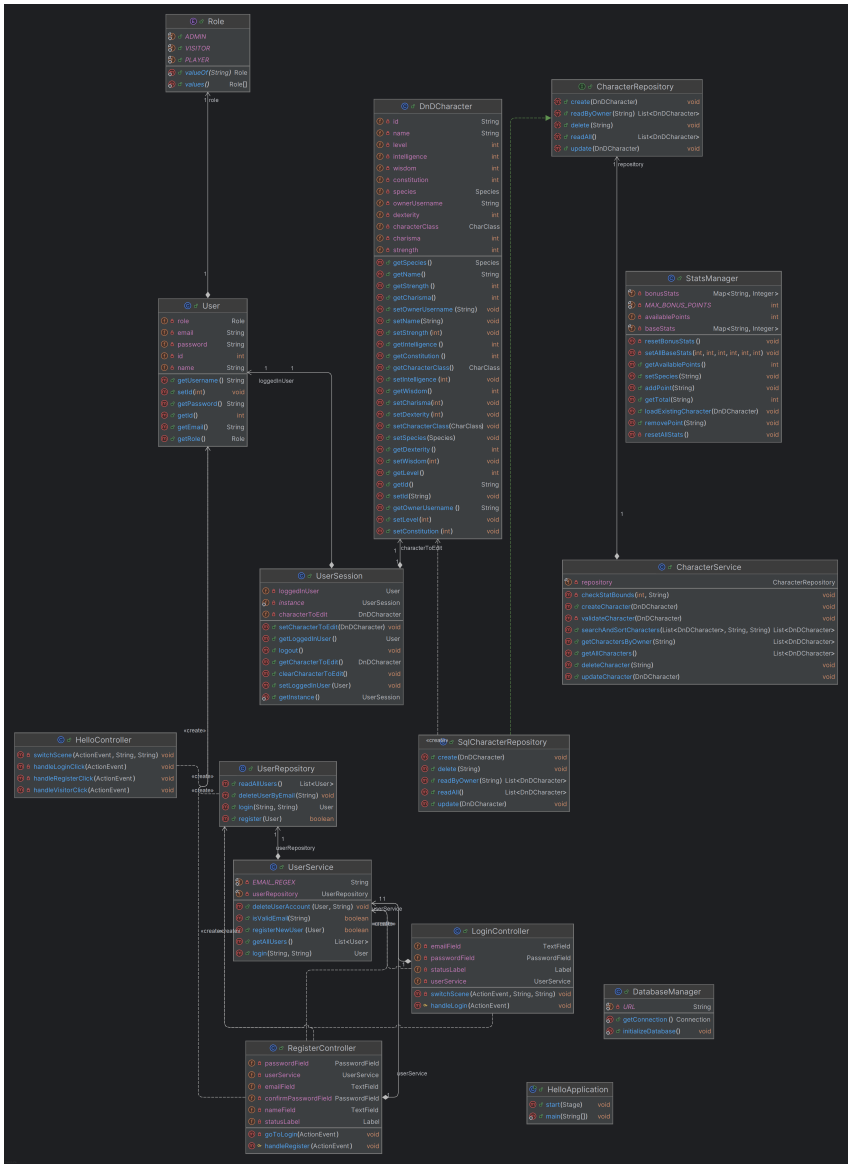

Figure 3: Class Diagram

3.7 Sequence diagrams

The sequence diagram illustrates how a user interacts with the system in the registration proccess.

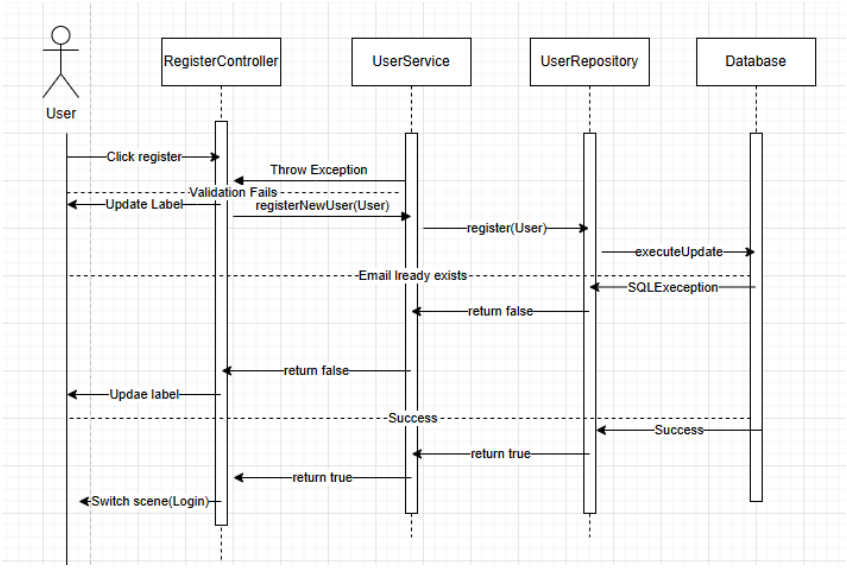

Figure 4: Register Sequence Diagram

4 Implementation phase - Application

4.1 Tools and technologies

The main tools and technologies used during the development are:

- **Programming Language :** Java, chosen for its strong object-oriented principles support for implementing the Model-View-Controller architecture.
- **GUI Framework :** JavaFX and FXML chosen to build a simple event-driven interface.
- **Database Management System:**SQLite was chosen for being a lightweight relational database.

4.2 Application structure

The application follows a strict architecture to separate data, logic, and presentation. The source code is organized under the root package, into three primary layers:

- `model` **(Data & Logic):** Contains the core domain entities User, DnDCharacter, UserSession, Role. It is subdivided into:
  - `service`**:** Handles business rules and data validation.
  - `repository`**:** Manages direct SQLite database operations.
  - `event`**:** Contains the interfaces and messaging objects for the Observer pattern.
  - `export`**:** Contains the Strategy pattern implementations for downloading files.
- `view` **(Presentation):** Contains all the JavaFX FXML files that define the graphical user interface.
- `controller` **(Bridge):** Contains the Java classes that intercept user actions from the view, communicate with the services, and update the UI.

4.3 User manual

The application features an intuitive, role-based Graphical User Interface (GUI). Upon launching, users start at the Welcome screen, where they can authenticate, register a new account or enter as a visitor.

Depending on the authenticated role, the interface automatically adapts its available features:

- **Visitor:** Can browse the application, can create, but cannot save, edit, or delete any data.
- **Player:** Can create new characters, edit their existing characters, and delete them from their catalog. They can also search and sort their existing characters.
- **Admin:** Can access a management dashboard to view all registered users and delete accounts if necessary.

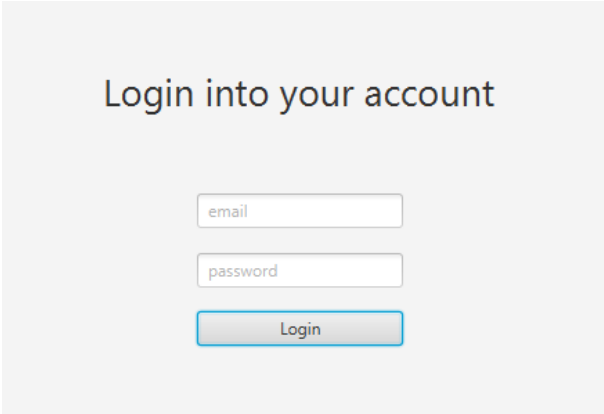

Figure 5: Login Page

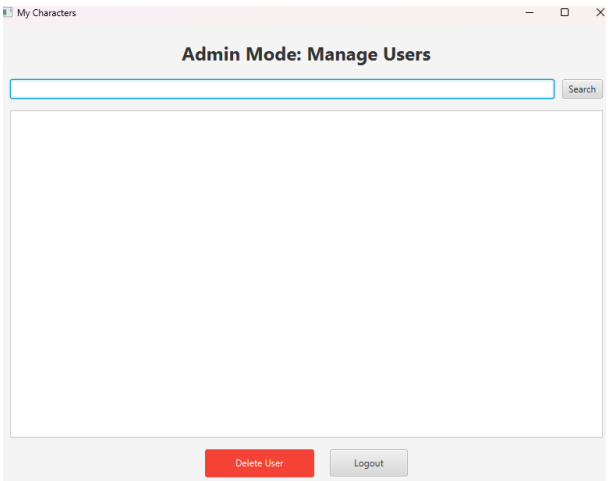

Figure 6: Admin Dashboard

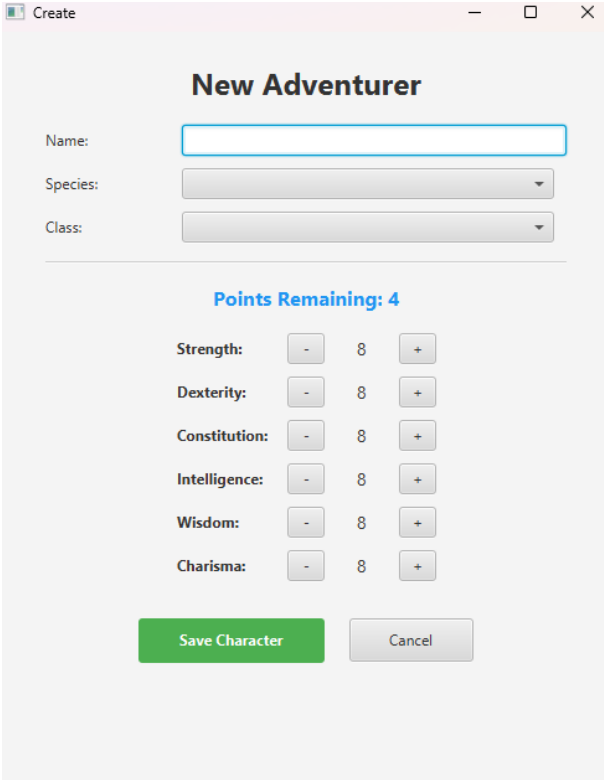

Figure 7: Create a Character Page

5 Testing and validation

Automated unit testing was implemented using the JUnit 5 framework. Testing focused on the application's core business logic to verify data integrity in isolation from the user interface. Specifically, test cases were developed for the UserService to ensure that the Regex responsible for email validation correctly accepts standard formats and strictly rejects invalid inputs before any database interaction occurs.